

MID を使用した解約率予測 GLM の改良 - 論文再現編

Reiji Kozuma^{*} Ryoichi Asashiba[†] Yoshihiro Matsumori[‡] Hirokazu Iwasawa[§]

2026-09-25

概要

本ドキュメントは研究の再現性確保の観点から、元論文 Kozuma et al. (2026) の主要な計算結果を再現するコードを提供するものである。層化モデルの構築や学習・検証ウィンドウの管理を担う補助コードを含む。複数か年にわたる大規模な計算を伴うため、計算に一定程度の負荷がかかることに注意が必要である。

目次

1 準備

- 1.1 パッケージとデータの準備
- 1.2 予測ワークフロー管理オブジェクトの設計
- 1.3 層化モデル関係の補助関数の用意
- 1.4 統計学的評価関数の統合および層化
- 1.5 EV 変動要因分析の関数化

2 モデルの構築

- 2.1 Reduced Baseline GLM
- 2.2 XGBoost Model
- 2.3 Baseline GLM
- 2.4 Improved GLM

3 精度比較

4 XGBoost のハイパーパラメータチューニング

5 基本編との整合性

6 検証環境

Acknowledgments

References

^{*} Daido Life Insurance Company, rktkdm@gmail.com

[†] Daido Life Insurance Company, ryoichi.asashiba@gmail.com

[‡] Sompo Japan Insurance Inc., ymatsumori1@sompo-japan.co.jp

[§] Graduate School of Accountancy, Waseda University, iwahiro@bb.mbn.or.jp

1 準備

1.1 パッケージとデータの準備

本ドキュメントで使用するパッケージを読み込む。

```
library(midr)
library(tidyverse)
library(readxl)
library(xgboost)
library(ROCR)
```

データの読み込みとその前処理、統計学的評価関数の設定は基本編と同一であるため、本ドキュメントでは記載を省略する。

読み込みの後、元論文で説明したような Rolling Window を作成する。purrr パッケージを利用し、year 列をキーとするデータフレームに学習・検証データを格納する。

```
years <- tibble(year = 2002:2010)

dfs_split <- years %>% mutate(
  df_train = map(year, \(year)
    df_all %>% filter(lapse_study_year >= year-2, lapse_study_year <= year)),
  df_test = map(year, \(year)
    df_all %>% filter(lapse_study_year == year+1)
  ))
```

データフレームから必要な列のみを取り出す部分を関数化する。

```
# Convert to design matrix
get_modelmatrix <- function(df){
  df <- prep_toint_pjratio(df)
  df <- model.matrix(~ ., df) # An intercept column of all 1s is created
}

get_xo <- function(df){ # All predictors and target variables
  df %>% select(setdiff(names(df),
    c("ID", "lapse_study_year", "policy_year", "issue_year",
      "exposure_amount")))
}

get_xe <- function(df){ # Excluding y (lapse rate)
  df <- df %>% get_xo
  df <- df %>% select(setdiff(names(df), c("lapse_rate")))
}

get_xy <- function(df){
  df <- df %>% get_xo
  df <- df %>% select(setdiff(names(df), c("lapse", "exposure")))
}

get_x <- function(df){ # Predictors only
  df <- df %>% get_xo
  df <- df %>% select(setdiff(names(df), c("lapse", "exposure", "lapse_rate")))
}
```

```

get_y <- function(df){ # y (lapse rate) only
  df %>% pull(lapse_rate)
}
get_w <- function(df){ # w (exposure) only
  df %>% pull(exposure)
}

```

1.2 予測ワークフロー管理オブジェクトの設計

本研究ではモデル構築とその評価を多数繰り返す。この手続きを独自のオブジェクトを用いて管理することでコードの見通しを良くする。tidymodels の workflows パッケージと考え方がよく似ているが、parsnip パッケージに対応したモデルでなくとも簡素に利用可能なよう、シンプルな実装としている。

メモリ節約の観点から、R 言語の参照クラス機能を利用して実装する。

```

Workflow <- setRefClass(
  Class = "Workflow",

  fields = list (
    fit_extra_arg = "list",
    prepfun       = "function", # function(newdata)
    fitfun        = "function", # function(df_train_prep, fit_extra_arg)
    predfun       = "function", # function(model_fitted, newdata_prep)
    model_fitted  = "ANY"
  ),

  methods = list (
    initialize = function(prepfun = function(x) x, fitfun, predfun) {
      prepfun <- prepfun
      fitfun <- fitfun
      predfun <- predfun
    },
    fit = function(df_train){
      tmp <- fitfun(prepfun(df_train), fit_extra_arg)
      model_fitted <- tmp
    },
    predict = function(newdata){
      predfun(model_fitted, prepfun(newdata))
    },
    get_predfun = function(){ # function(wf, newdata)
      function(wf, newdata){
        wf$predict(newdata)
      }
    },
    clone = function(){
      new_wf <- Workflow$new(prepfun, fitfun, predfun)
      new_wf
    }
  )
)

```

Rolling Window に亘って一括でモデル構築を行うためのユーティリティ関数を用意する。引数 wf_base

にはひな型となるオブジェクトを与える。全 Window に対する構築済みモデルがデータフレームに格納されて返却される。

```
fit_allwindow <- function(wf_base, dfs = dfs_split){
  dfs %>% transmute(year,
    wf = pmap(as.list(.), \(year, df_train, ...){
      wf <- wf_base$clone()
      message("fitting ",year," ", appendLF = FALSE)
      wf$fit(df_train)
      message("done")
      wf
    })
  )
}
```

fit_allwindow() 関数で返却されたデータフレームから、Workflow オブジェクトを一つ取り出すためのユーティリティ関数を用意する。

```
pluck_wf <- function(wfs, y){
  wfs %>% filter(year == y) %>% pull(wf) %>% first
}
```

1.3 層化モデル関係の補助関数の用意

本研究ではモデルを層化する形で実装する都合上、モデルの利用の際には適切な層にデータを分離する前処理が必要となる。データを層別に分離し、層別に個別の処理を繰り返す場合、purrr パッケージの機能を用いるのが便利である。ここではこのパッケージの機能を最大限活用することとし、他のパッケージによる関数も同様の考え方で機能を拡張する。

dplyr パッケージに実装されている group_nest(), group_by(), join_by() 関数について、層化に使用する変数を複数個とれるようにした拡張版を追加する。以下の関数では、引数 group に c(aa,bb) のような形で変数名を与えることで、それらによって層化された処理を行える。

```
group_nest_vec <- function(df = NULL, group = NULL, ...){
  vars_quo <- enquo(group)
  vars_expr <- rlang::quo_get_expr(vars_quo)
  if (is.null(vars_expr)){
    return(df %>% group_nest(...))
  }else{
    is_vec <- is.call(vars_expr) && vars_expr[[1]] == as.name("c")
    if (!is_vec) {
      return(df %>% group_nest({{group}}), ...)
    }
    var_syms <- as.list(vars_expr[-1]) # Extract contents of c(...)
    return(df %>% group_nest(!!!var_syms, ...))
  }
}
group_by_vec <- function(df = NULL, group = NULL, ...){
  vars_quo <- enquo(group)
```

```

vars_expr <- rlang::quo_get_expr(vars_quo)
if (is.null(vars_expr)){
  return(df %>% group_by(...))
}else{
  is_vec <- is.call(vars_expr) && vars_expr[[1]] == as.name("c")
  if (!is_vec) {
    return(df %>% group_by({{group}}, ...))
  }
  var_syms <- as.list(vars_expr[-1])
  return(df %>% group_by(!!!var_syms, ...))
}
}
join_by_vec <- function(group, ...){
  vars_quo <- enquos(group)
  vars_expr <- rlang::quo_get_expr(vars_quo)

  if (is.null(vars_expr)){
    return(join_by(...))
  }else{
    is_vec <- is.call(vars_expr) && vars_expr[[1]] == as.name("c")
    if (!is_vec) {
      return(join_by({{group}}, ...))
    }
    var_syms <- as.list(vars_expr[-1])
    return(join_by(!!!var_syms, ...))
  }
}
}

```

層化された処理の結果はデータフレームで得られる。そのようなデータフレームが複数あり、それらを結合したい場合は `join_by()` 関数を使用する。一方で、層化されていない場合は単にデータフレーム同士を `cbind()` すればよい。この双方の場合について同じ書き方で対応できるようなユーティリティ関数を用意する。

```

full_join_n <- function(x, y, group, ...){
  vars_quo <- enquos(group)
  vars_expr <- rlang::quo_get_expr(vars_quo)
  if (is.null(vars_expr)){
    cbind(x, y)
  }else{
    full_join(x, y, by = join_by_vec({{group}}), ...)
  }
}
}

```

1.4 統計学的評価関数の統合および層化

単一のモデル (Workflow オブジェクト) に対して、複数の統計学的評価結果をデータフレームに格納するユーティリティ関数を用意する。層化処理、すなわち層別に評価関数を適用する機能も用意するが、モデル自体が層化されている必要はない。引数 `types` に、`"logloss_total"`, `"logloss_mean"`, `"auc"` のどれかを設定したリストを指定する。

```

calc_scores <- function(wf, types, df_train = NULL, df_test = NULL, verbose = F, group =
  ↪ NULL){
  #browser()
  res <- NULL
  extras <- NULL
  y_pred <- NULL
  scores_logloss <- NULL
  for(type in types){
    score <- NA
    extra <- NA
    if (type == "logloss_mean"){
      type <- "logloss_mean_test"
    }
    if (type == "logloss_total"){
      type <- "logloss_total_test"
    }
    if (type == "auc"){
      type <- "auc_test"
    }

    if (type %in% c("logloss_mean_test", "logloss_total_test") &
        is.null(y_pred)){
      y_pred <- wf$predict(df_test)
      y_act <- df_test %>% get_y
      w <- df_test %>% get_w
      df_test <- df_test %>% mutate(.pred = y_pred, .act = y_act, .w = w)
    }

    if (type == "logloss_mean_test" || type == "logloss_total_test"){
      mean <- type == "logloss_mean_test"
      score <- df_test %>% group_nest_vec({{group}}) %>%
        mutate(.score = map_dbl(data, function(data){
          calc_logloss(data$.pred, data$.act, data$.w, mean = mean)
        })) %>%
        select(-data)
    }
    if (type == "auc_test"){
      tmp <- df_test %>% group_nest_vec({{group}}) %>%
        mutate(.tmp = map(data, function(data){
          auc <- calc_auc(data$.pred, data$.act, data$.w, verbose = TRUE)
          score <- as.numeric(auc$auc@y.values)
          list(auc = auc, score = score)
        })) %>% mutate(.score = map_dbl(.tmp, \(x) x$score), .auc = map(.tmp, \(x) x$auc))
      ↪ %>%
        group_by_vec({{group}})
      score <- tmp %>% select(group_vars(tmp), .score)

      if (verbose)
        extra <- tmp %>% select(group_vars(tmp), .auc)
    }
  }

```

```

score <- score %>% rename(!type := .score)
# extra <- wrap_atomic(extra, group = {{group}})
if (is.null(res)){
  res <- score
}else{
  res <- res %>% full_join_n(score, group = {{group}})
}
if (is.null(extras)){
  extras <- extra
}else{
  extras <- extras %>% full_join_n(extra, group = {{group}})
}
}
tmp <- extras %>% group_nest_vec({{group}})
res %>% full_join_n(tmp %>% rename(extras = data), group = {{group}})
}

```

1.5 EV 変動要因分析の関数化

fn_calc_ev() 関数の実装までは EV 編と同様であるので省略する。以降の変動要因分析を行う部分について、より効率的に行えるように関数を追加する。

まず、各シナリオの計算結果をサマライズする関数を用意する。

```

calc_evma_sub <- function(result, y, scenario,
                           df_sep, model, predfun, group = NULL,
                           verbose_return_all = F, verbose_ev_detail = F, ...){

  message("calculating EV scenario ", y, " ", scenario)
  ev <- fn_calc_ev(df_sep, model, predfun, ...)
  if (verbose_ev_detail){
    result$df_ev_detail <- result$df_ev_detail %>%
      rbind(cbind(y = y, scenario = scenario, df_sep, ev = ev$ev))
  }
  if (verbose_return_all){
    result$df_tmp_all <- ev
  }
  result$scenario <- rbind(result$scenario, tibble(y = y, scenario = scenario))

  result$ev <- rbind(result$ev,
                     cbind(tibble(y = y, scenario = scenario),
                             df_sep %>% mutate(ev = ev$ev) %>%
                               group_by_vec({{group}}) %>%
                               summarise(ev = sum(ev), .groups = "drop")))

  result
}

```

次に、指定した Rolling Window にわたってこの関数を複数回実行し、変動要因分析を実施する関数を実装する。

```

calc_evma_allwindow <- function(wfs, verbose = 0, group = NULL, dfs = dfs_split){
  ret <- list()
  ret$ev <- tibble()
  ret$ma <- tibble()
  ret$scenario <- tibble()
  verbose_ev_detail <- verbose >= 1
  if (verbose_ev_detail){
    ret$df_ev_detail <- tibble()
  }
  verbose_ev_cfdetail <- verbose >= 2

  for(i in 1:(length(wfs$year))){
    if (i > 1){
      y_start <- y_end
      wf_start <- wf_end
      df_start <- df_end
      ret_start <- ret_end
    }
    y_end <- wfs$year[[i]]
    wf_end <- wfs %>% filter(year == y_end) %>% pull(wf) %>% first
    df_end <- dfs %>% filter(year == y_end) %>% pull(df_test) %>% first %>%
      fn_sep_to_estimated

    if (i > 1){

      df_start_pred <- df_start %>%
        cbind(pred = ret_start$df_projeted_data %>% filter(t == 0) %>% select(lapse_rate)
        ↪ %>% pull) %>%
        mutate(exposure = exposure * (1 - pred)) %>%
        mutate(exposure_amount = exposure_amount * (1 - pred)) %>%
        select(-pred) %>% fn_shift_year(year = 1)

      ret <- calc_evma_sub(ret, y_end, "A",
        df_start_pred, wf_start, wf_start$get_predfun(),
        verbose_ev_detail = verbose_ev_detail, group = {{group}},
        ↪ verbose = 0)

      df_start_act <- df_start %>% fn_get_port_eopy

      ret <- calc_evma_sub(ret, y_end, "B",
        df_start_act, wf_start, wf_start$get_predfun(),
        verbose_ev_detail = verbose_ev_detail, group = {{group}},
        ↪ verbose = 0)

      ret <- calc_evma_sub(ret, y_end, "C",
        df_start_act, wf_end, wf_end$get_predfun(),
        verbose_ev_detail = verbose_ev_detail, group = {{group}},
        ↪ verbose = 0)
    }
    ret <- calc_evma_sub(ret, y_end, "End",

```

```

        df_end, wf_end, wf_end$get_predfun(),
        verbose_ev_detail = verbose_ev_detail, verbose_return_all = T,
        ↪ group = {{group}},
        verbose = if_else(verbose_ev_cfdetail, 2, 1))

ret_end <- ret$df_tmp_all
ret$df_tmp_all <- NULL

if (i > 1){
  df_start <- ret$scenario %>% filter(y == y_start, scenario == "End") %>%
    mutate(y = y_end, scenario = "0")
  df_tmp <- rbind(df_start, ret$scenario %>% filter(y == y_end))

  df_ev2 <- ret$ev %>% filter(y == y_start, scenario == "End") %>%
    mutate(y = y_end, scenario = "0")
  ret$ma <- rbind(ret$ma, df_ev2)
  for(r in 1:(nrow(df_tmp)-1)){
    df_ev1 <- df_ev2

    s1 <- df_tmp$scenario[[r]]
    s2 <- df_tmp$scenario[[r+1]]

    df_ev1 <- df_ev2 %>% rename(ev1 = ev) %>% select(-y, -scenario)
    df_ev2 <- ret$ev %>% filter(y == y_end, scenario == s2)
    df_ev12 <- df_ev1 %>% full_join_n(df_ev2, group = {{group}}) %>%
      mutate(ev = ev - ev1) %>% select(-y, -scenario, -ev1)
    df_ev12 <- cbind(
      tibble(y = y_end, scenario = paste0(s2,"-",s1)),
      df_ev12
    )
    ret$ma <- rbind(ret$ma, df_ev12)
  }
}
ret
}

```

さらに、前述の統計学的評価関数を計算する機能をこの関数に統合する。

```

types_all <- c("logloss_mean", "logloss_total", "auc", "B-A", "C-B")
setting_types_evma <- c("B-A", "C-B")

calc_scores_allwindow <- function(wfs, types, verbose = FALSE, group = NULL, dfs =
  ↪ dfs_split){

  types_mvma <- intersect(types, setting_types_evma)
  types_nonmvma <- setdiff(types, setting_types_evma)

  res <- list()
  df <- NULL
  if (length(types_nonmvma) > 0){

```

```

df <- dfs %>% inner_join(wfs, by = join_by(year == year)) %>%
  transmute(
    year,
    result = pmap(list(wf = wf, df_train = df_train, df_test = df_test),
      calc_scores, types = types_nonmvma, verbose = verbose, group =
↪ {{group}})#
    ) %>% unnest(result)
  if (!verbose){
    df <- df %>% select(-extras)
  }
}

if (length(types_mvma) > 0){
  evma <- calc_evma_allwindow(wfs, ifelse(verbose, 1, 0), group = {{group}}, dfs = dfs)
  if (TRUE)#if (verbose)
    res$evma <- evma
  df_evma <- evma$ma %>% group_by_vec(group = {{group}})
  df_evma <- df_evma %>% mutate(year = y - 1) %>%
    pivot_wider(names_from = scenario, values_from = ev) %>%
    select(group_vars(df_evma), all_of(c("year", types_mvma)))

  if (is.null(df)){
    df <- df_evma
  }else{
    df <- df %>% full_join(df_evma, by = join_by_vec(group = {{group}}, year == year))
    if (verbose){
      df <- df %>% select(-extras, everything(), extras)
    }

  }
}
res$scores <- df
res
}

```

calc_scores_allwindow() 関数には EV 変動要因分析における各シナリオの EV をデータフレームの形で出力する機能がある。これを見やすく表示するための関数を定義する。

```

fn_get_evma <- function(scores_ev){
  scores_ev <- scores_ev %>% mutate(
    year = case_when(
      scenario == "End" ~ y,
      scenario == "A" ~ y-1,
      scenario == "B" ~ y-1,
      scenario == "C" ~ y-1
    ),
    scenario = case_when(
      scenario == "End" ~ "y-End",
      scenario == "A" ~ "y+1-A",
      scenario == "B" ~ "y+1-B",

```

```

    scenario == "C" ~ "y+1-C"
  )
)
scores_ev_s <- scores_ev %>% filter(
  year < max(scores_ev$year)
)
scores_ev_e <- scores_ev %>% filter(scenario == "y-End") %>% mutate(
  year = year - 1,
  scenario = "y+1-End"
) %>% filter(
  year >= min(scores_ev$year)
)
scores_ev_ma <- rbind(scores_ev_s, scores_ev_e) %>% select(-y) %>%
  pivot_wider(names_from = scenario, values_from = ev) %>% mutate(
    `A-End` = `y+1-A` - `y-End`,
    `B-A` = `y+1-B` - `y+1-A`,
    `C-B` = `y+1-C` - `y+1-B`,
    `End-C` = `y+1-End` - `y+1-C`,
  )

tab_show <- scores_ev_ma %>%
  transmute(
    `Evaluation Year` = year,
    `y-End` = `y-End`,
    `(y+1)-A` = `y+1-A`,
    `(y+1)-B` = `y+1-B`,
    `(y+1)-C` = `y+1-C`,
    `(y+1)-End` = `y+1-End`,
    `A-End` = `A-End`,
    `B-A` = `B-A`,
    `C-B` = `C-B`,
    `End-C` = `End-C`
  )

tab_show
}

```

2 モデルの構築

2.1 Reduced Baseline GLM

2.1.1 前処理の実装

SOA の作成した GLM と同様の前処理を行う関数を作成する。ただし、ここで構築するのは `face_amount` を取り除いた Reduced Baseline GLM である。後で `face_amount` を戻した Baseline GLM も構築するので、処理が共通する部分は関数を分離して実装する。

```

prepfun_soa_common <- function(df_tmp){
  df_tmp %>% prep_toint_age %>% prep_toint_pjratio %>%
    mutate(
      issue_age_sq = issue_age^2, issue_age_log = log(issue_age),

```

```

risk_class_group = risk_class %>%
  factor(levels = levels_risk_class,
    labels = c("Super-Pref NS", "NS", "NS", "NS", "NS", "SM", "SM", "SM", "SM")),
premium_mode_group = premium_mode %>%
  factor(levels = levels_premium_mode,
    labels = c("Annual", "Semiannual/Quarterly", "Semiannual/Quarterly",
      "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
premium_jump_ratio_group = cut(
  if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
  breaks = c(0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 16, 20, 99), right = FALSE))
}

prepfun_soared_xy <- function(df_tmp){
  df_tmp %>%
  prepfun_soa_common %>%
  select(duration, post_level_premium_structure,
    issue_age, issue_age_sq, issue_age_log, risk_class_group,
    premium_mode_group, premium_jump_ratio_group, exposure, lapse)
}

```

次に基本編と同様、モデルに投入するデータに対する前処理を実装する。ただし、今回は各 Validation Year ごとに異なるモデルを構築しなければならないうえに、層化モデルのそれぞれに対しても個別に前処理を定める必要がある。モデルごとにきめ細やかに前処理方法を定める必要があるため、コードは複雑となる。

`premium_jump_ratio_group` は、学習データで現れないカテゴリを検出し、振替先カテゴリへの対応表を用意する。振替先は、より高いカテゴリの中で一番近いカテゴリ、それが無い場合はより低いカテゴリの中で一番近いカテゴリとする。

他の説明変数も同じように対応表を用意する。`risk_class_group` は、"Super-Pref NS" が学習データで現れないことがあるため、性質の近い "NS" へ振り替える。`premium_mode_group` も "Unknown/Other" が学習データで現れないことがある。この群団は比較的解約率が高いことから、他の群団で最も解約率の高い "Annual" へ振り替える。

```

prep_emptylevel_soa <- function(data){
  model <- list()

  # Identify zero-count levels in premium_jump_ratio_group and set replacement levels
  df_alt <- tibble()
  tb <- table(data$premium_jump_ratio_group)
  for(rg in list(length(tb):1, 1:length(tb))){
    alt <- NULL
    for(i in rg){
      if (tb[i] == 0){
        if (!is.null(alt)){
          df_alt <- rbind(df_alt, tibble(org = names(tb)[i], alt = alt))
          tb[i] <- -1
        }
      }else{
        alt <- names(tb)[i]
      }
    }
  }
}

```

```

    }
  }
  model$df_alt_premium_jump_ratio_group <- df_alt

  # Identify zero-count levels in risk_class_group and set replacement levels
  df_alt <- tibble()
  tb <- table(data$risk_class_group)
  if (tb[["Super-Pref NS"]] == 0){
    df_alt <- rbind(df_alt, tibble(org = "Super-Pref NS", alt = "NS"))
  }
  model$df_alt_risk_class_group <- df_alt

  # Identify zero-count levels in premium_mode_group and set replacement levels
  df_alt <- tibble()
  tb <- table(data$premium_mode_group)
  if (tb[["Unknown/Other"]] == 0){
    df_alt <- rbind(df_alt, tibble(org = "Unknown/Other", alt = "Annual"))
  }
  model$df_alt_premium_mode_group <- df_alt

  model
}

```

2.1.2 層別モデル構築部分の実装

層化したモデルの実装にあたり、各モデルの実体はデータフレームに格納される形とする。そして `tidyr::nest()` と `purrr::map()` を組み合わせることで、データをグループごとに分割し、それぞれ処理を行った結果をデータフレームに格納する。

まずは、各層におけるモデルの構築を行う関数を用意する。

```

fitfun_soa_st <- function(data, duration, post_level_premium_structure, ...){
  model <- prep_emptylevel_soa(data)

  model$model <- glm(cbind(lapse, exposure - lapse) ~
    (. + premium_jump_ratio_group * issue_age) - 1,
    family = binomial(), data = data)
  gc(reset = TRUE)

  model
}

```

続けて、学習データを各層に分離し、上記のモデル構築関数に受け渡す部分を実装する。ここに `duration` と `post_level_premium_structure` によるデータの層分けを記述している。

```

fitfun_soa_factory <- function(fitfun_st)
  function(train, fit_extra_arg){
    model <- list()
    model_soa <- train %>% group_nest(duration, post_level_premium_structure) %>%
      transmute(duration, post_level_premium_structure,
        model = pmap(as.list(.), fitfun_st))
  }
}

```



```

      mutate(premium_jump_ratio_group2 = coalesce(alt, premium_jump_ratio_group2)) %>%
      select(-alt)
    }

    df_alt <- model_st$df_alt_risk_class_int #I-6
    if (!is.null(df_alt) && nrow(df_alt) > 0){
      data <- data %>%
        left_join(df_alt,
                  join_by(risk_class_int == org)) %>%
        mutate(risk_class_int = coalesce(alt, risk_class_int)) %>%
        select(-alt)
    }

    tmp <- model_st$risk_class_int_null #I-6
    if (!is.null(tmp) && tmp){
      data <- data %>% select(-risk_class_int)
    }

    data
  }

```

次に、各層で具体的に予測を行う部分を実装する。

```

predfun_soa_st <- function(model, data, duration, post_level_premium_structure, ...){
  d <- duration
  p <- post_level_premium_structure

  model_st <- model %>% filter(
    duration == d, post_level_premium_structure == p
  ) %>% pull(model) %>% first

  data <- bake_emptylevel_soa(model_st, data)
  model_obj <- model_st$model

  predict(
    model_obj,
    data %>% select(-.rowid),
    type = c("response"))
}

```

最後に、予測したいデータを各層に分離し、各層の予測関数に受け渡す部分を実装する。ここで注意が必要なのは、予測結果は層別に分離する前のデータフレームと同じ順序で返却しなければならないことである。`.rowid` という列に当初の順序を記録しておき、予測統合後にこの列でソートすることで順序を再現することとする。

```

predfun_soa <- function(model, newdata){
  newdata %>%
    mutate(.rowid = row_number()) %>%
    group_nest(duration, post_level_premium_structure) %>%
    mutate(

```

```

    .pred = pmap(as.list(.), predfun_soa_st, model = model$model)
  ) %>%
  unnest(cols = c(data, .pred)) %>% arrange(.rowid) %>% pull(.pred)
}

```

2.1.4 モデルの構築と精度評価

ここまでで、モデル構築の一連の手順を全て関数の形で実装できた。これらを最初に用意した Workflow オブジェクトに装備し、`fit_allwindow()` 関数を適用することで、全 Window にわたる全ての学習過程が一括で行われる。

```

wf_base_soared <- Workflow$new(prepfun_soared_xy, fitfun_soa, predfun_soa)
wfs_soared <- fit_allwindow(wf_base_soared)

```

こうして得られた全 Window のモデル群に `calc_scores_allwindow()` 関数を適用することで、統計学的評価関数による評価と EV 変動要因分析の双方がすべての Window で計算される。

```

scores_soared <- calc_scores_allwindow(wfs_soared, types = types_all, verbose = T)

```

全てのスコアを表示すると次のとおり。なお、この表の `year` は統計学的評価関数の場合は Validation Year-1 が、EV 変動要因分析の場合は Evaluation Year に対応する。

```

scores_soared$scores %>% select(-extras)

```

```

# A tibble: 9 x 6
  year logloss_mean_test logloss_total_test auc_test `B-A` `C-B`
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 2002      0.29219      87095. 0.68717 -214759. 1975852.
2 2003      0.27930     141620. 0.66194 -434247. -36123110.
3 2004      0.27874     169340. 0.71064  605118. -20131480.
4 2005      0.29002     208324. 0.79599 -1248435. -10254700.
5 2006      0.29859     237976. 0.80653 -4599537. -29279590.
6 2007      0.29500     268504. 0.84393 -8842926.  9616194.
7 2008      0.29638     255877. 0.84403 -9728781. -32820937.
8 2009      0.30758     251568. 0.86216 -7259208. -28134626.
9 2010      0.31717     215470. 0.87131      NA      NA

```

加えて、元論文で表示した完全な変動要因分析を出力する。`calc_scores_allwindow()` 関数には各シナリオの EV を副次的に出力する機能がある。これを表示すると次のとおり。

```

scores_soared$evma$ev %>% fn_get_evma

```

```

# A tibble: 8 x 10
  `Evaluation Year` `y-End` `(y+1)-A` `(y+1)-B` `(y+1)-C` `(y+1)-End`
<dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1      2002 88768661. 86186046. 85971287. 87947139. 200999017.
2      2003 200999017. 204043777. 203609530. 167486420. 217515784.

```

3	2004	217515784.	216328297.	216933416.	196801936.	259063446.
4	2005	259063446.	252342497.	251094062.	240839362.	298981824.
5	2006	298981824.	289454006.	284854470.	255574880.	342110882.
6	2007	342110882.	321888687.	313045762.	322661956.	379131596.
7	2008	379131596.	361291992.	351563211.	318742274.	370972576.
8	2009	370972576.	351257698.	343998490.	315863864.	335040958.
		`A-End`	`B-A`	`C-B`	`End-C`	
		<dbl>	<dbl>	<dbl>	<dbl>	
1		-2582615.	-214759.	1975852.	113051878.	
2		3044759.	-434247.	-36123110.	50029364.	
3		-1187487.	605118.	-20131480.	62261510.	
4		-6720950.	-1248435.	-10254700.	58142462.	
5		-9527817.	-4599537.	-29279590.	86536003.	
6		-20222195.	-8842926.	9616194.	56469640.	
7		-17839604.	-9728781.	-32820937.	52230302.	
8		-19714878.	-7259208.	-28134626.	19177093.	

2.2 XGBoost Model

続けて、XGBoost のモデル構築を行う。モデル構築および予測の具体的実装は基本編と同様である。また、これを Workflow オブジェクトに装備し、モデルの構築と精度評価を行う手続きは Reduced Baseline GLM と同様である。

ハイパーパラメータ等の設定は次のとおり。

```
params_xgb <- list(
  objective = "reg:logistic",
  eval_metric = "logloss", # Used in CV
  # Hyper parameters. Determined by Bayesian optimization using the "tune" package.
  gamma = 2.966439e-08, # loss_reduction
  eta = 0.02, # learn_rate
  max_depth = 10, # tree_depth
  min_child_weight = 10, # min_n
  subsample = 0.9976594, # sample_size
  colsample_bynode = 6/31 # mtry / count of predictors
)
SETTING_DEBUG_XGB <- FALSE # If TRUE, display log during training
SETTING_XGB_NROUND <- -1 # If -1, dynamically determined by CV during training
```

まず、モデル構築を行う関数を実装する。この中には、学習データですべての行で同じ数値となっている列の記録と、交差検証による `nrounds` の決定も含まれる。

```
fitfun_xgb <- function(train, fit_extra_arg){
  df_train_x_tmp <- train %>% get_x %>% get_modelmatrix

  constant_cols <- which(apply(df_train_x_tmp, 2, function(col) all(col == col[1])))
  dm_train <- xgb.DMatrix(data = df_train_x_tmp[, -constant_cols, drop = FALSE],
```


1	2002	0.28978	86377.	0.69745	701730.	7112249.
2	2003	0.27771	140813.	0.66686	1091596.	-18447009.
3	2004	0.27752	168599.	0.71648	2931168.	9174755.
4	2005	0.28645	205761.	0.80581	1351642.	15074741.
5	2006	0.29355	233957.	0.82060	-1806857.	-8728909.
6	2007	0.29981	272878.	0.84636	-9263861.	-40450363.
7	2008	0.29167	251818.	0.85053	-7441370.	-38172620.
8	2009	0.29887	244443.	0.86806	-4414217.	-27545209.
9	2010	0.31239	212221.	0.87749	NA	NA

```
scores_xgb$evma$ev %>% fn_get_evma
```

```
# A tibble: 8 x 10
```

	`Evaluation Year`	`y-End`	`(y+1)-A`	`(y+1)-B`	`(y+1)-C`	`(y+1)-End`
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2002	68852189.	63730019.	64431748.	71543997.	145765359.
2	2003	145765359.	139756693.	140848289.	122401281.	161612923.
3	2004	161612923.	151680473.	154611641.	163786396.	225880839.
4	2005	225880839.	213799882.	215151524.	230226265.	300033231.
5	2006	300033231.	290905775.	289098917.	280370008.	386946478.
6	2007	386946478.	373056003.	363792142.	323341779.	379513856.
7	2008	379513856.	364924183.	357482812.	319310193.	369927370.
8	2009	369927370.	352041613.	347627396.	320082187.	339330329.

	`A-End`	`B-A`	`C-B`	`End-C`
	<dbl>	<dbl>	<dbl>	<dbl>
1	-5122170.	701730.	7112249.	74221362.
2	-6008666.	1091596.	-18447009.	39211642.
3	-9932450.	2931168.	9174755.	62094443.
4	-12080957.	1351642.	15074741.	69806966.
5	-9127456.	-1806857.	-8728909.	106576470.
6	-13890474.	-9263861.	-40450363.	56172077.
7	-14589673.	-7441370.	-38172620.	50617177.
8	-17885757.	-4414217.	-27545209.	19248142.

2.3 Baseline GLM

修正 I-1 により `face_amount` を復元した「**Baseline GLM**」, つまり SOA が構築したものと同等のモデルをこれまでと同様の手続きで構築する。前処理のための関数をごく一部修正するだけでよい。

```
prepfun_soa_xy <- function(df_tmp){
  df_tmp %>%
    prepfun_soa_common %>%
```

```
select(duration, post_level_premium_structure,
       issue_age, issue_age_sq, issue_age_log, risk_class_group,
       premium_mode_group, premium_jump_ratio_group, face_amount, exposure, lapse)
}
```

Workflow オブジェクトの作成とモデル構築を行う。

```
wf_base_soa <- Workflow$new(prepfun_soa_xy, fitfun_soa, predfun_soa)
wfs_soa <- fit_allwindow(wf_base_soa)
```

評価指標を計算すると次のとおり。

```
scores_soa <- calc_scores_allwindow(wfs_soa, types = types_all, verbose = T)
```

```
scores_soa$scores %>% select(-extras)
```

A tibble: 9 x 6

	year	logloss_mean_test	logloss_total_test	auc_test	`B-A`	`C-B`
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2002	0.29219	87095.	0.68929	686060.	-3510132.
2	2003	0.27916	141551.	0.66310	1108221.	-28327486.
3	2004	0.27906	169537.	0.71157	1942614.	-16782788.
4	2005	0.29007	208358.	0.79685	117519.	5436455.
5	2006	0.29822	237676.	0.80732	-1388917.	-16457593.
6	2007	0.29443	267985.	0.84449	-6019893.	2225231.
7	2008	0.29585	255421.	0.84459	-6298941.	-31923367.
8	2009	0.30704	251125.	0.86230	-4241520.	-16470645.
9	2010	0.31669	215138.	0.87173	NA	NA

```
scores_soa$evma$ev %>% fn_get_evma
```

A tibble: 8 x 10

	`Evaluation Year`	`y-End`	`(y+1)-A`	`(y+1)-B`	`(y+1)-C`	`(y+1)-End`
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2002	75471873.	72032707.	72718767.	69208634.	171245174.
2	2003	171245174.	170991910.	172100131.	143772645.	187522826.
3	2004	187522826.	183051081.	184993696.	168210908.	223044826.
4	2005	223044826.	212087087.	212204606.	217641062.	270928745.
5	2006	270928745.	258839658.	257450741.	240993147.	323560356.
6	2007	323560356.	302505963.	296486071.	298711302.	352732870.
7	2008	352732870.	332133296.	325834355.	293910988.	343637815.
8	2009	343637815.	321216098.	316974578.	300503933.	319154485.
	`A-End`	`B-A`	`C-B`	`End-C`		
	<dbl>	<dbl>	<dbl>	<dbl>		

```

1 -3439166.    686060.   -3510132. 102036540.
2  -253264.   1108221.  -28327486.  43750181.
3  -4471745.   1942614. -16782788.  54833918.
4 -10957739.   117519.    5436455.  53287684.
5 -12089087. -1388917. -16457593.  82567209.
6 -21054393. -6019893.   2225231.  54021568.
7 -20599573. -6298941. -31923367.  49726827.
8 -22421717. -4241520. -16470645.  18650552.

```

2.4 Improved GLM

元論文における 10 個の改良を全て施したモデル「Improved GLM」の構築方法は次のとおり。基本編で示したものとほぼ同様であるものの、全 Window の各層でも対応できるように一部の実装が異なる。

```

prepfun_mod_common <- function(df_tmp){
  df_tmp %>% prep_toint_age %>% prep_toint_pjratio %>%
  mutate(
    premium_mode_oo = #I-4
    if_else(
      duration >= "10" & post_level_premium_structure == "Other" &
        premium_mode == "Unknown/Other", 1, 0),
    post_level_premium_structure = if_else( #I-4
      premium_mode_oo == 1, "ART", post_level_premium_structure),
    premium_mode = if_else( #I-4
      premium_mode_oo == 1, "Monthly", premium_mode),
    issue_age_sq = issue_age^2, issue_age_log = log(issue_age),
    issue_age_70 = if_else(issue_age >= 70 & duration == "06-09", 1, 0), #I-5
    risk_class_group = risk_class %>% #I-2
      factor(levels = levels_risk_class,
        labels = c("Super-Pref NS", "NS", "NS", "NS", "Undiff NS",
          "SM", "SM", "Undiff SM", "Agg/Unknown")),
    risk_class_int = factor( #I-6
      if_else(
        (risk_class == "Undiff SM" | risk_class == "Non-Pref SM" |
          risk_class == "Undiff NS" | risk_class == "Non-Pref NS") &
          duration == "06-09" |
          (risk_class == "Undiff NS" | risk_class == "Non-Pref NS") &
            duration == "10" |
            risk_class == "Undiff NS" ,
          risk_class, "Super-Pref NS"),
        levels = levels_risk_class),
    premium_mode_group = premium_mode %>% #I-3
      factor(levels = levels_premium_mode,
        labels = c("Annual", "Semiannual", "Quarterly",
          "Monthly/Biweekly", "Monthly/Biweekly", "Unknown/Other")),
    premium_mode_group_q = if_else( #I-3
      duration == "10" & premium_mode == "Quarterly", 1, 0),
    premium_jump_ratio_group = cut(
      if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),

```

```

    breaks = c(0, 1, 2, 3, 4, 5, 6, 7, 8, 10, 12, 16, 20, 99), right = FALSE),
  premium_jump_ratio_group2 = cut( #I-3
    if_else(premium_jump_ratio_unknown == 1, 0, premium_jump_ratio),
    breaks = c(0, 1, 2, 5, 99), right = FALSE),
  premium_jump_ratio_rc_NPNS = if_else( #I-7
    risk_class == "Non-Pref NS", premium_jump_ratio, 0),
  premium_jump_ratio_rc_PBNS = if_else( #I-7
    risk_class == "Pref Best NS" & (duration >= "10"), premium_jump_ratio, 0),
  face_amount_D_prun = if_else( #I-9
    duration == "06-09" & post_level_premium_structure == "ART" &
    premium_jump_ratio_unknown == 1 & face_amount == "1M-",
    1, 0))
}

prepfun_mod_xy <- function(df_tmp){
  df_tmp %>%
    prepfun_mod_common %>%
    select(duration, post_level_premium_structure,
      issue_age, issue_age_sq, issue_age_log, risk_class_group,
      issue_age_70, #I-5
      risk_class_int, #I-6
      premium_mode_group, premium_jump_ratio_group,
      premium_jump_ratio_group2, premium_mode_group_q, #I-3
      premium_jump_ratio_rc_NPNS, premium_jump_ratio_rc_PBNS, #I-7
      face_amount, #I-1
      face_amount_D_prun, #I-9
      gender, #I-10
      exposure, lapse)
}

```

特に、学習データに存在しないか、件数が少ないカテゴリへの対応が非常に複雑になる。

```

prep_mod_threshold_risk_class_group <- 700 #I-2
prep_mod_threshold_risk_class_group_SP <- 50 #I-2
prep_mod_threshold_risk_class_group_UKN <- 100 #I-2
prep_mod_threshold_risk_class_int <- 1000 #I-6
prep_mod_threshold_premium_jump_ratio_rc_NPNS <- 300 #I-7
prep_mod_threshold_premium_jump_ratio_rc_PBNS <- 100 #I-7
prep_mod_threshold_face_amount_D_prun <- 500 #I-9

prep_emptylevel_mod <- function(data){
  model <- prep_emptylevel_soa(data)

  # Identify zero-count levels in risk_class_group and set replacement levels #I-2
  df_alt <- tibble()
  tb <- table(data$risk_class_group)
  if (tb[["Super-Pref NS"]] <= prep_mod_threshold_risk_class_group_SP){
    df_alt <- rbind(df_alt, tibble(org = "Super-Pref NS", alt = "NS"))
  }
  if (tb[["Agg/Unknown"]] <= prep_mod_threshold_risk_class_group_UKN){
    df_alt <- rbind(df_alt, tibble(org = "Agg/Unknown", alt = "SM"))
  }
}

```

```

}
if (tb[["SM"]] <= prep_mod_threshold_risk_class_group){
  df_alt <- rbind(df_alt, tibble(org = "SM", alt = "Undiff SM")) %>%
    mutate(alt = if_else(alt == "SM", "Undiff SM", alt))
}
if (tb[["NS"]] <= prep_mod_threshold_risk_class_group){
  df_alt <- rbind(df_alt, tibble(org = "NS", alt = "Undiff NS")) %>%
    mutate(alt = if_else(alt == "NS", "Undiff NS", alt))
}
if (tb[["Undiff NS"]] <= prep_mod_threshold_risk_class_group){
  df_alt <- rbind(df_alt, tibble(org = "Undiff NS", alt = "NS")) %>%
    mutate(alt = if_else(alt == "Undiff NS", "NS", alt))
}

model$df_alt_risk_class_group <- df_alt

#I-3
df_ <- data %>% group_nest(premium_mode_group_q) %>%
  mutate(df = map(data, function(data){
    df_alt <- tibble()
    tb <- table(data$premium_jump_ratio_group2)
    for(rg in list(length(tb):1, 1:length(tb))){
      alt <- NULL
      for(i in rg){
        if (tb[i] == 0){
          if (!is.null(alt)){
            df_alt <- rbind(df_alt, tibble(org = names(tb)[i], alt = alt))
            tb[i] <- -1
          }
        }else{
          alt <- names(tb)[i]
        }
      }
    }
    df_alt
  })) %>% select(-data) %>% unnest(df)
model$df_alt_premium_jump_ratio_group2_st <- df_

# Identify zero-count levels in risk_class_int and set replacement levels #I-6
df_alt <- tibble()
tb <- table(data$risk_class_int)
if (tb[["Undiff SM"]] <= prep_mod_threshold_risk_class_int){
  df_alt <- rbind(df_alt, tibble(org = "Undiff SM", alt = "Super-Pref NS"))
}
if (tb[["Non-Pref SM"]] <= prep_mod_threshold_risk_class_int){
  df_alt <- rbind(df_alt, tibble(org = "Non-Pref SM", alt = "Super-Pref NS"))
}
if (tb[["Non-Pref NS"]] <= prep_mod_threshold_risk_class_int){
  df_alt <- rbind(df_alt, tibble(org = "Non-Pref NS", alt = "Super-Pref NS"))
}

```

```

if (tb[["Undiff NS"]] <= prep_mod_threshold_risk_class_int){
  df_alt <- rbind(df_alt, tibble(org = "Undiff NS", alt = "Super-Pref NS"))
}
if (nrow(df_alt) > 0){
  data_tmp <- data %>%
    left_join(df_alt,
              join_by(risk_class_int == org)) %>%
    mutate(risk_class_int = coalesce(alt, risk_class_int))
  model$risk_class_int_null <- length(unique(data_tmp$risk_class_int)) == 1
}
model$df_alt_risk_class_int <- df_alt

model
}

```

モデル構築部分の関数は次のとおり。glm() 関数に引き渡す formula も、Window や層によって変化させる必要がある。

```

fitfun_mod_st <- function(data, duration, post_level_premium_structure, ...){
  model <- prep_emptylevel_mod(data)
  data <- bake_emptylevel_soa(model, data)

  tmp <- ""
  if (!is.null(model$risk_class_int_null) && model$risk_class_int_null){ #I-6
    tmp <- paste0(tmp, "")
  }else{
    tmp <- paste0(tmp, " + risk_class_int * issue_age")
  }
  if (sum(data$premium_jump_ratio_rc_NPNS != 0) >=
        prep_mod_threshold_premium_jump_ratio_rc_NPNS){ #I-7
    tmp <- paste0(tmp, "")
  }else{
    tmp <- paste0(tmp, " - premium_jump_ratio_rc_NPNS")
  }
  if (sum(data$premium_jump_ratio_rc_PBNS != 0) >=
        prep_mod_threshold_premium_jump_ratio_rc_PBNS){ #I-7
    tmp <- paste0(tmp, "")
  }else{
    tmp <- paste0(tmp, " - premium_jump_ratio_rc_PBNS")
  }
  if (sum(data$face_amount_D_prun) < prep_mod_threshold_face_amount_D_prun){ #I-9
    data <- data %>% select(-face_amount_D_prun)
  }
  if ((duration == "06-09" || duration == "10") & #I-8
        post_level_premium_structure == "ART"){
    tmp <- paste0(tmp, " + issue_age : face_amount")
  }
  formula_string_additional <- tmp

  formula_string <- paste0(

```

```

"cbind(lapse, exposure - lapse) ~ (. ",
"- premium_jump_ratio_group2 - premium_mode_group_q ", #I-3
"+ premium_jump_ratio_group2 : premium_mode_group_q", #I-3
tmp,
"+ premium_jump_ratio_group * issue_age ) - 1")

model$model <- glm(as.formula(formula_string),
  family = binomial(), data = data)
model
}
fitfun_mod <- fitfun_soa_factory(fitfun_mod_st)

```

Workflow オブジェクトの作成とモデル構築はこれまでと同様.

```

wf_base_mod <- Workflow$new(prepfun_mod_xy, fitfun_mod, predfun_soa)
wfs_mod <- fit_allwindow(wf_base_mod)

```

評価指標を計算すると次のとおり.

```
scores_mod <- calc_scores_allwindow(wfs_mod, types = types_all, verbose = T)
```

```
scores_mod$scores %>% select(-extras)
```

A tibble: 9 x 6

	year	logloss_mean_test	logloss_total_test	auc_test	`B-A`	`C-B`
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2002	0.29199	87036.	0.69322	959670.	-3112986.
2	2003	0.27903	141482.	0.66342	2114609.	-10970565.
3	2004	0.27919	169614.	0.71385	2266373.	-22597775.
4	2005	0.28919	207729.	0.79886	932605.	8125765.
5	2006	0.29824	237691.	0.80908	-1482747.	-27154354.
6	2007	0.29338	267026.	0.84605	-5336373.	14246869.
7	2008	0.29505	254733.	0.84619	-6426290.	-21617963.
8	2009	0.30690	251013.	0.86382	-3597573.	-787664.
9	2010	0.31591	214608.	0.87385	NA	NA

```
scores_mod$evma$ev %>% fn_get_evma
```

A tibble: 8 x 10

	`Evaluation Year`	`y-End`	`(y+1)-A`	`(y+1)-B`	`(y+1)-C`	`(y+1)-End`
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2002	76441059.	73090728.	74050397.	70937412.	156165262.
2	2003	156165262.	153941989.	156056598.	145086033.	189737920.
3	2004	189737920.	185500911.	187767284.	165169510.	220799283.
4	2005	220799283.	209524927.	210457533.	218583297.	271025355.

5	2006	271025355.	258723114.	257240367.	230086014.	307961423.
6	2007	307961423.	284643955.	279307583.	293554452.	340976781.
7	2008	340976781.	319954211.	313527921.	291909958.	336803039.
8	2009	336803039.	313197464.	309599891.	308812227.	326401508.
	`A-End`	`B-A`	`C-B`	`End-C`		
	<dbl>	<dbl>	<dbl>	<dbl>		
1	-3350331.	959670.	-3112986.	85227850.		
2	-2223272.	2114609.	-10970565.	44651887.		
3	-4237009.	2266373.	-22597775.	55629773.		
4	-11274355.	932605.	8125765.	52442057.		
5	-12302240.	-1482747.	-27154354.	77875410.		
6	-23317468.	-5336373.	14246869.	47422329.		
7	-21022570.	-6426290.	-21617963.	44893081.		
8	-23605574.	-3597573.	-787664.	17589281.		

3 精度比較

`calc_scores_allwindow()` 関数で得られたすべての評価結果に対して、他のモデルとの比較を表示するためのユーティリティ関数を用意する。引数 `calc.mean` を `TRUE` とすると、全 Window の結果をサマライズした RMSE または算術平均値を計算する。

```
compare_scores <- function(ls_scores, calc.mean = F, pivot = F, group = NULL){
  cols <- Reduce(intersect, map(ls_scores, names))
  df_results <- tibble(modelname = names(ls_scores), data = ls_scores) %>%
    mutate(data = map(data, function(n){
      select(n, all_of(cols))
    })) %>%
    unnest(data)

  if (pivot){
    df_results <- df_results %>% group_by_vec(group = {{group}})
    df_results <- df_results %>% pivot_wider(id_cols = c(year, group_vars(df_results)),
      names_from = modelname,
      values_from = setdiff(cols, c("year",
        ⇨ group_vars(df_results))))
  }

  if (calc.mean){
    if ("extras" %in% names(df_results)){
      df_results <- df_results %>% select(-extras)
    }
    df_results <- df_results %>% group_by_vec(group = {{group}}, modelname) %>%
      summarise(across(c(-year),
        function(v)if(cur_column() %in% c("B-A", "C-B")){
          mean(v^2, na.rm = T)^0.5
        }else{mean(v)}), .groups = "drop")
  }
}
```

```

    }

    df_results
  }

```

これを用いて 4 モデルの比較を出力すると次のとおり。元論文の結果が再現されている。

```

ls_scores <- list()
ls_scores[["soa_reduced"]] <- scores_soared$scores %>% select(-extras)
ls_scores[["soa_baseline"]] <- scores_soa$scores %>% select(-extras)
ls_scores[["soa_improved"]] <- scores_mod$scores %>% select(-extras)
ls_scores[["xgb"]] <- scores_xgb$scores %>% select(-extras)

compare_scores(ls_scores, calc.mean = T)

```

A tibble: 4 x 6

	modelname	logloss_mean_test	logloss_total_test	auc_test	`B-A`	`C-B`
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	soa_baseline	0.29474	203765.	0.78792	3558823.	18344522.
2	soa_improved	0.29432	203437.	0.78982	3469760.	16259691.
3	soa_reduced	0.29500	203975.	0.78708	5577385.	24077363.
4	xgb	0.29197	201874.	0.79440	4691027.	24058817.

```
compare_scores(ls_scores, pivot = T)
```

A tibble: 9 x 21

	year	logloss_mean_test_soa_reduced	logloss_mean_test_soa_baseline
	<dbl>	<dbl>	<dbl>
1	2002	0.29219	0.29219
2	2003	0.27930	0.27916
3	2004	0.27874	0.27906
4	2005	0.29002	0.29007
5	2006	0.29859	0.29822
6	2007	0.29500	0.29443
7	2008	0.29638	0.29585
8	2009	0.30758	0.30704
9	2010	0.31717	0.31669

	logloss_mean_test_soa_improved	logloss_mean_test_xgb
	<dbl>	<dbl>
1	0.29199	0.28978
2	0.27903	0.27771
3	0.27919	0.27752
4	0.28919	0.28645
5	0.29824	0.29355

6	0.29338	0.29981	
7	0.29505	0.29167	
8	0.30690	0.29887	
9	0.31591	0.31239	
	logloss_total_test_soa_reduced	logloss_total_test_soa_baseline	
	<dbl>	<dbl>	
1	87095.	87095.	
2	141620.	141551.	
3	169340.	169537.	
4	208324.	208358.	
5	237976.	237676.	
6	268504.	267985.	
7	255877.	255421.	
8	251568.	251125.	
9	215470.	215138.	
	logloss_total_test_soa_improved	logloss_total_test_xgb	auc_test_soa_reduced
	<dbl>	<dbl>	<dbl>
1	87036.	86377.	0.68717
2	141482.	140813.	0.66194
3	169614.	168599.	0.71064
4	207729.	205761.	0.79599
5	237691.	233957.	0.80653
6	267026.	272878.	0.84393
7	254733.	251818.	0.84403
8	251013.	244443.	0.86216
9	214608.	212221.	0.87131
	auc_test_soa_baseline	auc_test_soa_improved	auc_test_xgb
	<dbl>	<dbl>	<dbl>
1	0.68929	0.69322	0.69745
2	0.66310	0.66342	0.66686
3	0.71157	0.71385	0.71648
4	0.79685	0.79886	0.80581
5	0.80732	0.80908	0.82060
6	0.84449	0.84605	0.84636
7	0.84459	0.84619	0.85053
8	0.86230	0.86382	0.86806
9	0.87173	0.87385	0.87749
	``B-A_soa_reduced``		
	<dbl>	<dbl>	<dbl>
1	686060.	959670.	701730.
2			
3			
4			
5			
6			
7			
8			
9			

2	1108221.	2114609.	1091596.	-36123110.
3	1942614.	2266373.	2931168.	-20131480.
4	117519.	932605.	1351642.	-10254700.
5	-1388917.	-1482747.	-1806857.	-29279590.
6	-6019893.	-5336373.	-9263861.	9616194.
7	-6298941.	-6426290.	-7441370.	-32820937.
8	-4241520.	-3597573.	-4414217.	-28134626.
9	NA	NA	NA	NA
	`C-B_soa_baseline`	`C-B_soa_improved`	`C-B_xgb`	
	<dbl>	<dbl>	<dbl>	
1	-3510132.	-3112986.	7112249.	
2	-28327486.	-10970565.	-18447009.	
3	-16782788.	-22597775.	9174755.	
4	5436455.	8125765.	15074741.	
5	-16457593.	-27154354.	-8728909.	
6	2225231.	14246869.	-40450363.	
7	-31923367.	-21617963.	-38172620.	
8	-16470645.	-787664.	-27545209.	
9	NA	NA	NA	

4 XGBoost のハイパーパラメータチューニング

元論文に記載のとおり、ハイパーパラメータは Bayesian optimization によりチューニングする。具体的な実装には、tidymodels に含まれる tune パッケージを使用する。典型的なモデルに対するハイパーパラメータチューニングを比較的簡単なコードで行える。

最初に、ハイパーパラメータチューニングに使用するデータセットを指定する。ここでは、2005 年から 2007 年の学習データを用いることとする。ここで決定したハイパーパラメータは、ローリングウィンドウ全体を通じて固定される。

```
library(tidymodels)
y <- 2007 # Training data used for hyperparameter tuning
df_train_tmp <- dfs_split %>% filter(year == y) %>% pull(df_train) %>% first
df_test_tmp <- dfs_split %>% filter(year == y) %>% pull(df_test) %>% first

# Prepare preprocessed training data for XGBoost
df_train_x_xgb <- df_train_tmp %>% get_x %>% get_modelmatrix

# Apply importance_weights or relevant weighting function to weights column beforehand
df_train_xgb <- cbind(as_tibble(df_train_x_xgb),
                      y = df_train_tmp %>% get_y,
                      w = df_train_tmp %>% get_w %>% importance_weights)

# Prepare test data for XGBoost
df_test_x_xgb <- df_test_tmp %>% get_x %>% get_modelmatrix
```

チューニングの際の評価指標は Log-loss とするが、yardstick パッケージに備わるデフォルトの実装 `mn_log_loss` は、`boost_tree(mode = "regression")` の場合に用いることができないため、カスタムメトリックを作成して対応する。

```
#https://www.tidymodels.org/learn/develop/metrics/
logloss_impl <- function(truth, estimate, case_weights = NULL) {
  calc_logloss(estimate, truth, case_weights)
}

logloss_vec <- function(data, truth, estimate, na_rm = TRUE, case_weights = NULL, ...) {
  check_numeric_metric(truth, estimate, case_weights)
  if (na_rm) {
    result <- yardstick_remove_missing(truth, estimate, case_weights)
    truth <- result$truth
    estimate <- result$estimate
    case_weights <- result$case_weights
  } else if (yardstick_any_missing(truth, estimate, case_weights)) {
    return(NA_real_)
  }
  logloss_impl(truth, estimate, case_weights = case_weights)
}

library(rlang)
logloss <- function(data, ...) {
  UseMethod("logloss")
}

logloss <- new_numeric_metric(logloss, direction = "minimize")
logloss.data.frame <- function(data, truth, estimate, na_rm = TRUE, case_weights = NULL,
  ↪ ...) {
  numeric_metric_summarizer(
    name = "logloss",
    fn = logloss_vec,
    data = data,
    truth = !!enquo(truth),
    estimate = !!enquo(estimate),
    na_rm = na_rm,
    case_weights = !!enquo(case_weights)
  )
}
```

そして、次のコードで実際に最適なハイパーパラメータの探索を行う。learning rate は 0.02 に固定し、ブースティングの反復回数は学習のたびに early stopping により決定する。そのほかのハイパーパラメータを、Bayesian optimization により決定する。

```
# Model specification
engine_tune_xgb <- boost_tree(mode = "regression") %>%
  set_engine(
    "xgboost",
    objective = "reg:logistic",
    eval_metric = "logloss",
```

```

# Proportion of validation data used for early stopping (remaining used for training)
validation = 0.2
) %>%
set_args(
  # Early stopping rounds (stop if no improvement for 10 consecutive iterations)
  stop_iter = 10,
  # nrounds for xgboost
  trees = 2000,
  loss_reduction = tune::tune(), # gamma for xgboost
  learn_rate = 0.02, #tune::tune(), # eta for xgboost
  tree_depth = tune::tune(), # max_depth for xgboost
  min_n = tune::tune(), # min_child_weight for xgboost
  sample_size = tune::tune(), # subsample for xgboost
  mtry = tune::tune() # colsample_bynode for xgboost
)

# Hyperparameter search space specification
params_tune <- list(
  #trees(),
  loss_reduction(), tree_depth(), min_n(), #learn_rate(),
  sample_size = sample_prop(),
  mtry() %>% dials::finalize(df_train_xgb)
) %>%
dials::parameters()

# Tuning data setup (specify v for v-fold cross-validation)
set.seed(2025)
splitscv_df_train_xgb <- vfold_cv(df_train_xgb, v = 4)

# Parallel processing setup: not supported on Windows with custom metrics
#library(future)
#plan(multisession)
#cat("Number of parallel workers: ", nbrOfWorkers(), "\n")
t1 <- proc.time() # Measure computation time

# Execute tuning with the above specifications
wf_tune_xgb <- workflow(y ~ .) %>%
  add_model(engine_tune_xgb)

set.seed(2025)
tunedresult_xgb <- wf_tune_xgb %>%
  tune_bayes(
    resamples = splitscv_df_train_xgb,
    param_info = params_tune,
    initial = 10, # Number of initial random candidate designs
    iter = 20, # Number of subsequent Bayesian optimization iterations
    metrics = yardstick::metric_set(logloss),
    control = control_bayes(verbose = TRUE, save_workflow = TRUE)
  )

```

```
t2 <- proc.time()
t <- (t2-t1)[3]
names(t) <- NULL
mtry_cols <- ncol(df_train_x_xgb) # Denominator for colsample_bynode
```

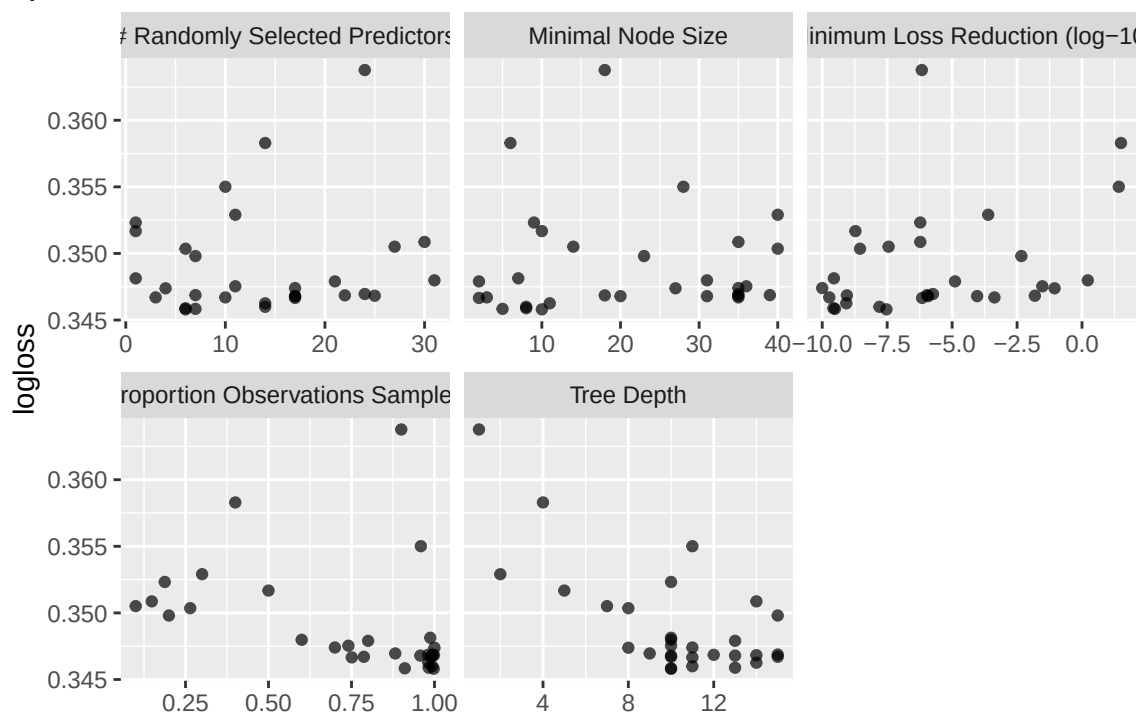
```
save(tunedresult_xgb, t, mtry_cols, file = SETTING_FNAME_TUNEDRESULT_INST_REPROD)
```

これにより、元の論文の結果が再現される。

```
cat("Tuning time:", t, "s\n")
cat("mtry_cols:", mtry_cols, "\n") # Denominator for colsample_bynode
print(autoplot(tunedresult_xgb))
best_xgb <- tune::select_best(tunedresult_xgb, metric = "logloss")
best_xgb
```

Tuning time: 955.5 s

mtry_cols: 31



A tibble: 1 x 6

	loss_reduction	tree_depth	min_n	sample_size	mtry	.config
	<dbl>	<int>	<int>	<dbl>	<int>	<chr>
1	2.9664e-8	10	10	0.99766	6	iter16

5 基本編との整合性

注意深い読者は、基本編の GLM の精度評価指標が元論文や本ドキュメントの数値と一致しないことに気づくだろう。本節はこの不一致について原因を確認するものである。たとえば、Baseline GLM について不

致の状況を確認すると次のとおりである。

```
load(SETTING_FNAME_MODELS_INST_BASIC)

year_midr <- 2006 # Validation Year - 1
df_train_2006 <- df_all %>%
  filter(lapse_study_year >= year_midr-2, lapse_study_year <= year_midr)
df_test_2006 <- df_all %>%
  filter(lapse_study_year == year_midr+1)

df_train_2006_soa <- df_train_2006 %>% prepfun_soa_xy
df_test_2006_soa <- df_test_2006 %>% prepfun_soa_xy

pred_soa_basic <- model_soa_2006 %>%
  predict(df_test_2006_soa %>% prepfun_soared_xy_test_2006, type = c("response"))

logloss_basic <- calc_logloss(
  pred_soa_basic,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = FALSE
)#237853.5

auc_basic <- calc_auc(
  pred_soa_basic,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure)
)#0.8073218

logloss_reprod <- scores_soa$scores %>% filter(year == year_midr) %>%
  ↪ pull(logloss_total_test)
auc_reprod <- scores_soa$scores %>% filter(year == year_midr) %>% pull(auc_test)
```

ROC-AUC は計算誤差程度の差しかないことから非常に近いモデルであることが示されている一方、Log-loss にわずかな差が見られることから、出力される確率の値に差が生じていることがわかる。

```
cat("Log-loss reprod:", logloss_reprod, ", basic:", logloss_basic,
    ", basic - reprod: ", logloss_basic - logloss_reprod, "\n")
```

Log-loss reprod: 237676.5 , basic: 237853.5 , basic - reprod: 177.0003

```
cat("ROC-AUC reprod:", auc_reprod, ", basic:", auc_basic,
    ", basic - reprod: ", auc_basic - auc_reprod)
```

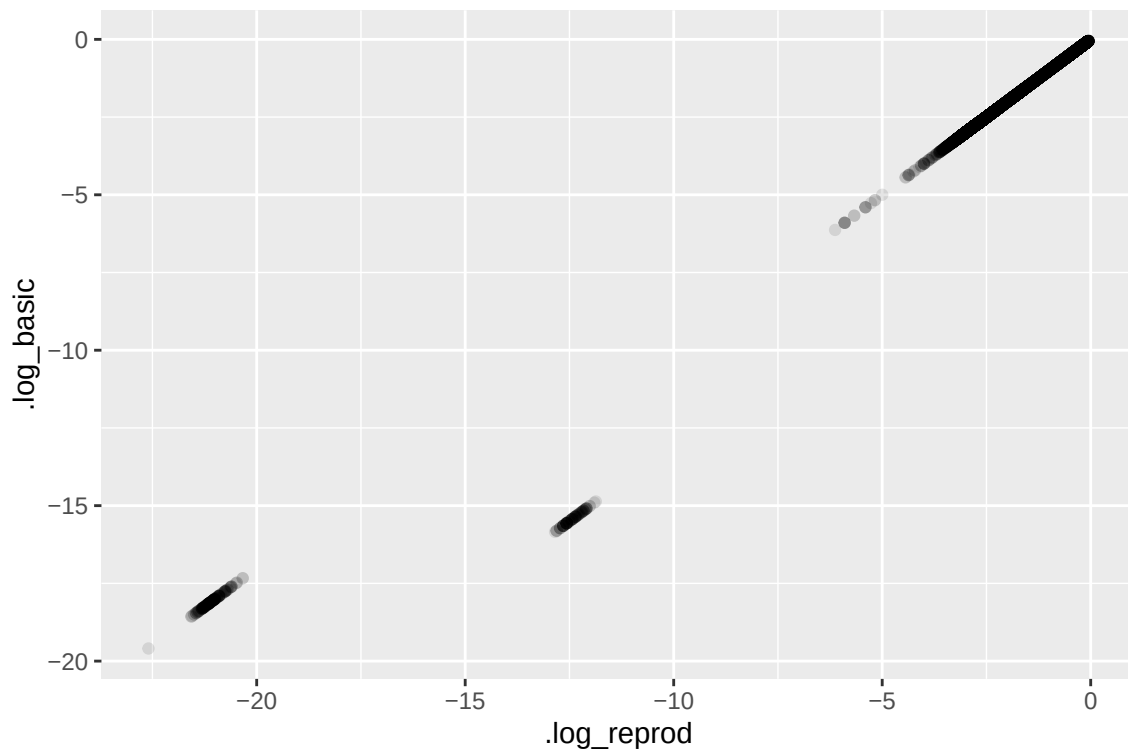
ROC-AUC reprod: 0.8073218 , basic: 0.8073218 , basic - reprod: -6.729719e-10

その原因は、本ドキュメントでは層ごとに別々のモデルを構築している一方、基本編では1つのモデルとして構築しているためである。両者は本質的に同等なモデルであるが、学習データにおいて1件も解約が発生しないケースの予測において、わずかに出力される確率値が異なっている。実際、両者の予測値の対数をそれぞれ縦軸、横軸にとってプロットすると次の図のようになり、大多数の予測値は一致している一方で、ゼロに近

い予測値を出力している箇所で差異が生まれていることが確認できる。

```
wf_soa_reprod <- wfs_soa %>% filter(year == year_midr) %>% pull(wf) %>% first
pred_soa_reprod <- wf_soa_reprod$predict(df_test_2006)

pred_soa_c <- cbind(df_test_2006, .log_reprod = log(pred_soa_reprod),
                    .log_basic = log(pred_soa_basic))
pred_soa_c %>% ggplot(mapping = aes(x = .log_reprod, y = .log_basic)) +
  geom_point(alpha = 0.1)
```



例えば .log_reprod が -15 以上 -10 未満となるような検証データがどのようなものを特定してみる。次のように、`post_level_premium_structure == "ART" & duration == 12 & premium_jump_ratio > 12` の条件であることがわかる。この GLM では `premium_jump_ratio` はいくつかの区分に分けてダミー変数化されており、この群団の解約率が対応する回帰係数に直接的に反映される。

```
pred_soa_c %>% filter(.log_reprod >= -15, .log_reprod <= -10) %>%
  group_by(post_level_premium_structure, duration, premium_jump_ratio) %>%
  summarise(sum(exposure))
```

A tibble: 7 x 4

Groups: post_level_premium_structure, duration [1]

	post_level_premium_structure	duration	premium_jump_ratio	`sum(exposure)`
	<fct>	<ord>	<fct>	<dbl>
1	ART	12	12.01-13.00	40
2	ART	12	13.01-14.00	21.530

3 ART	12	14.01-15.00	29
4 ART	12	15.01-16.00	48
5 ART	12	16.01-17.00	5
6 ART	12	17.01-18.00	3
7 ART	12	18.01-19.00	8

この条件は、学習データでは解約が1件も記録されていない。

```
df_train_2006 %>% filter(post_level_premium_structure == "ART", duration == 12) %>%
  group_by(premium_jump_ratio) %>% summarise(sum(exposure), sum(lapse))
```

A tibble: 13 x 3

	premium_jump_ratio	`sum(exposure)`	`sum(lapse)`
<fct>	<dbl>	<dbl>	<dbl>
1 " 1.01- 2.00"	2131.1	258	
2 " 2.01- 3.00"	1402.1	255	
3 " 3.01- 4.00"	644.12	109	
4 " 4.01- 5.00"	690.36	128	
5 " 5.01- 6.00"	434.17	87	
6 " 6.01- 7.00"	270.33	67	
7 " 7.01- 8.00"	247.44	53	
8 " 8.01- 9.00"	134.66	31	
9 " 9.01-10.00"	89.847	33	
10 "10.01-11.00"	21	6	
11 "11.01-12.00"	5	1	
12 "12.01-13.00"	1	0	
13 "Unknown"	1868.2	318	

さらに例えば .log_reprod が -20 未満となるような検証データがどのようなものを特定してみる。次のように、post_level_premium_structure == "Other" & duration == "06-09" & premium_mode == "Unknown/Other" の条件であることがわかる。この群団の解約率が対応する回帰係数に直接的に反映される。

```
pred_soa_c %>% filter(.log_reprod <= -20) %>%
  group_by(post_level_premium_structure, duration, premium_mode) %>%
  summarise(sum(exposure))
```

A tibble: 1 x 4

```
# Groups:   post_level_premium_structure, duration [1]
post_level_premium_structure duration premium_mode `sum(exposure)`
<fct>          <ord>      <fct>          <dbl>
1 Other          06-09    Unknown/Other    21082
```

この条件は、学習データでは解約が1件も記録されていない。

```
df_train_2006 %>% filter(post_level_premium_structure == "Other", duration == "06-09") %>%
  group_by(premium_mode) %>% summarise(sum(exposure), sum(lapse))
```

```
# A tibble: 5 x 3
  premium_mode `sum(exposure)` `sum(lapse)`
  <fct>          <dbl>          <dbl>
1 Annual          44542.          2261
2 Semiannual       9433.5           437
3 Quarterly        42138.         2333
4 Monthly         120400.         7175
5 Unknown/Other    16364             0
```

最後に、同じような関係が Reduced Baseline GLM と Improved GLM でも成立するかを確認する。Reduced Baseline GLM では次のとおりで、同じ関係が成立している。

```
df_train_2006_soared <- df_train_2006 %>% prepfun_soared_xy
df_test_2006_soared <- df_test_2006 %>% prepfun_soared_xy

pred_soared_basic <- model_soared_2006 %>%
  predict(df_test_2006_soared %>% prepfun_soared_xy_test_2006, type = c("response"))

logloss_basic <- calc_logloss(
  pred_soared_basic,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = FALSE
)

auc_basic <- calc_auc(
  pred_soared_basic,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure)
)

logloss_reprod <- scores_soared$scores %>% filter(year == year_midr) %>%
  pull(logloss_total_test)
auc_reprod <- scores_soared$scores %>% filter(year == year_midr) %>% pull(auc_test)
cat("Log-loss reprod:", logloss_reprod, ", basic:", logloss_basic,
    ", basic - reprod: ", logloss_basic - logloss_reprod, "\n")
```

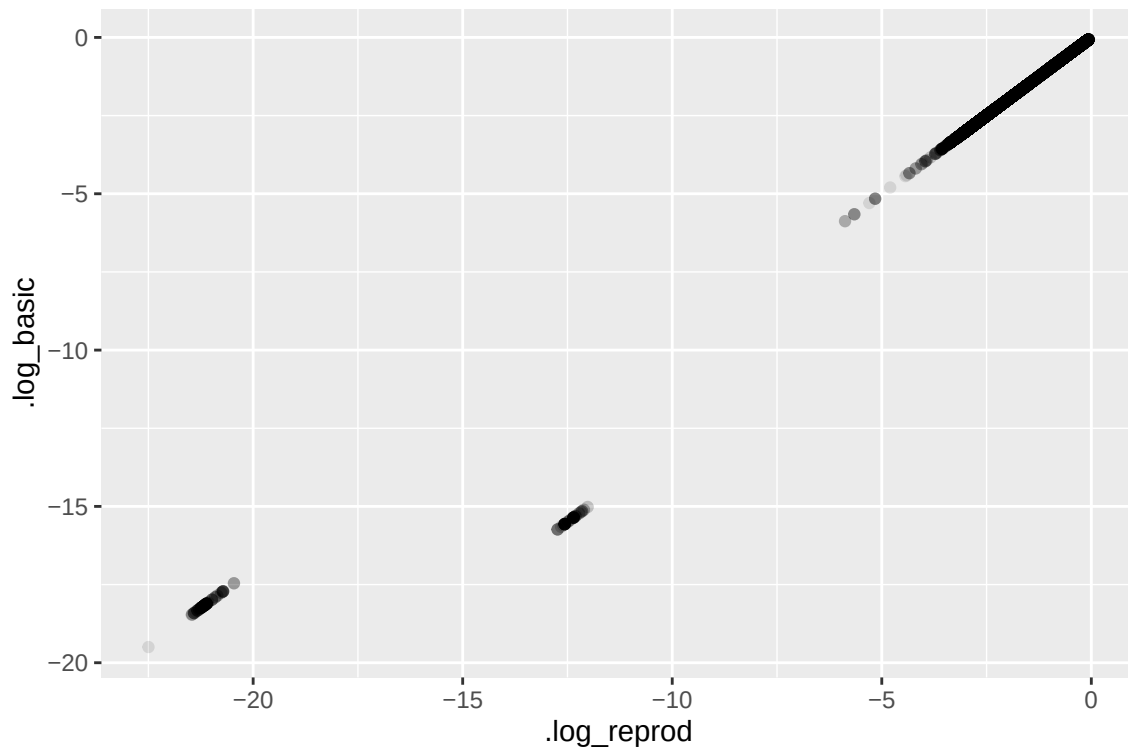
Log-loss reprod: 237976.1 , basic: 238153.1 , basic - reprod: 177.0003

```
cat("ROC-AUC reprod:", auc_reprod, ", basic:", auc_basic, ",
    basic - reprod: ", auc_basic - auc_reprod)
```

ROC-AUC reprod: 0.8065347 , basic: 0.8065347 ,
basic - reprod: 0

```
wf_soared_reprod <- wfs_soared %>% filter(year == year_midr) %>% pull(wf) %>% first
pred_soared_reprod <- wf_soared_reprod$predict(df_test_2006)
```

```
pred_soared_c <- cbind(df_test_2006, .log_reprod = log(pred_soared_reprod),
                      .log_basic = log(pred_soared_basic))
pred_soared_c %>% ggplot(mapping = aes(x = .log_reprod, y = .log_basic)) +
  geom_point(alpha = 0.1)
```



Improved GLM では次のとおりで、ここでも同じ関係が成立している。

```
df_train_2006_mod <- df_train_2006 %>% prepfun_mod_xy_basic
df_test_2006_mod <- df_test_2006 %>% prepfun_mod_xy_basic

pred_mod_basic <- model_mod_2006 %>%
  predict(df_test_2006_mod %>% prepfun_soared_xy_test_2006, type = c("response"))

logloss_basic <- calc_logloss(
  pred_mod_basic,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure),
  mean = FALSE
)

auc_basic <- calc_auc(
  pred_mod_basic,
  df_test_2006 %>% pull(lapse_rate),
  df_test_2006 %>% pull(exposure)
)
```

```
logloss_reprod <- scores_mod$scores %>% filter(year == year_midr) %>%
  ↪ pull(logloss_total_test)
auc_reprod <- scores_mod$scores %>% filter(year == year_midr) %>% pull(auc_test)
cat("Log-loss reprod:", logloss_reprod, ", basic:", logloss_basic,
    ", basic - reprod: ", logloss_basic - logloss_reprod, "\n")
```

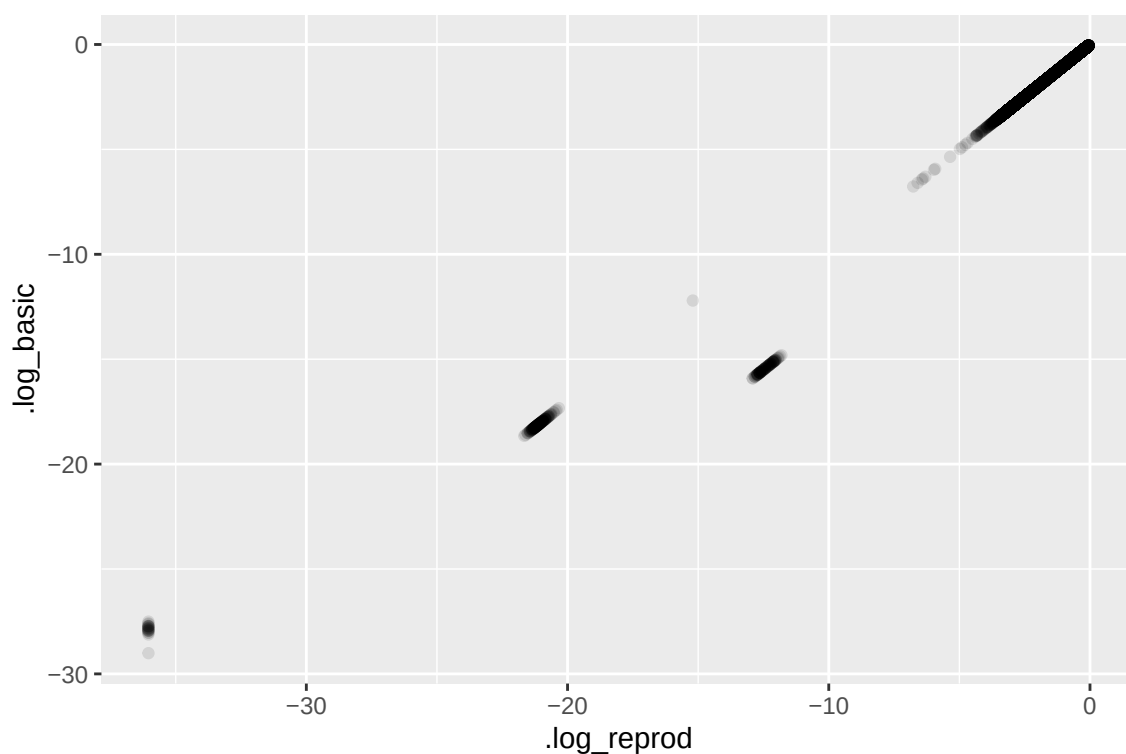
Log-loss reprod: 237690.7 , basic: 237867.7 , basic - reprod: 177.0004

```
cat("ROC-AUC reprod:", auc_reprod, ", basic:", auc_basic,
    ", basic - reprod: ", auc_basic - auc_reprod)
```

ROC-AUC reprod: 0.8090849 , basic: 0.8090849 , basic - reprod: -4.457337e-09

```
wf_mod_reprod <- wfs_mod %>% filter(year == year_midr) %>% pull(wf) %>% first
pred_mod_reprod <- wf_mod_reprod$predict(df_test_2006)

pred_mod_c <- cbind(df_test_2006, .log_reprod = log(pred_mod_reprod),
                    .log_basic = log(pred_mod_basic))
pred_mod_c %>% ggplot(mapping = aes(x = .log_reprod, y = .log_basic)) +
  geom_point(alpha = 0.1)
```



6 検証環境

R やパッケージのバージョンによって計算結果が異なる可能性がある。ここでは、本ドキュメントを作成するのに使用したパッケージ等のバージョンを記す。

```
sessionInfo()
```

```
R version 4.6.0 (2026-04-24 ucrt)
```

```
Platform: x86_64-w64-mingw32/x64
```

```
Running under: Windows 11 x64 (build 26200)
```

```
Matrix products: default
```

```
LAPACK version 3.12.1
```

```
locale:
```

```
[1] LC_COLLATE=Japanese_Japan.utf8 LC_CTYPE=Japanese_Japan.utf8
```

```
[3] LC_MONETARY=Japanese_Japan.utf8 LC_NUMERIC=C
```

```
[5] LC_TIME=Japanese_Japan.utf8
```

```
time zone: Asia/Tokyo
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] tune_2.1.0      waterfalls_1.1.4 readxl_1.5.0      insurance_0.2.0
```

```
[5] ROCR_1.0-12     xgboost_3.2.1.1 lubridate_1.9.5   forcats_1.0.1
```

```
[9] stringr_1.6.0   dplyr_1.2.1      purrr_1.2.2       readr_2.2.0
```

```
[13] tidyr_1.3.2     tibble_3.3.1     ggplot2_4.0.3     tidyverse_2.0.0
```

```
[17] midr_0.6.1
```

```
loaded via a namespace (and not attached):
```

```
[1] tidyselect_1.2.1 viridisLite_0.4.3 timeDate_4052.112
```

```
[4] farver_2.1.2      S7_0.2.2          fastmap_1.2.0
```

```
[7] digest_0.6.39     rpart_4.1.27      timechange_0.4.0
```

```
[10] lifecycle_1.0.5   yardstick_1.4.0    survival_3.8-6
```

```
[13] magrittr_2.0.5     compiler_4.6.0     rlang_1.2.0
```

```
[16] tools_4.6.0        utf8_1.2.6         yaml_2.3.12
```

```
[19] data.table_1.18.4  knitr_1.51         labeling_0.4.3
```

```
[22] DiceDesign_1.10    RColorBrewer_1.1-3 parsnip_1.6.0
```

```
[25] withr_3.0.3        workflows_1.3.0    nnet_7.3-20
```

```
[28] grid_4.6.0         future_1.70.0      globals_0.19.1
```

```
[31] scales_1.4.0       MASS_7.3-65        tinytex_0.60
```

```
[34] cli_3.6.6          rmarkdown_2.31     generics_0.1.4
```

[37]	otel_0.2.0	rstudioapi_0.19.0	future.apply_1.20.2
[40]	tzdb_0.5.0	splines_4.6.0	dials_1.4.4
[43]	parallel_4.6.0	cellranger_1.1.0	vctrs_0.7.3
[46]	hardhat_1.4.3	Matrix_1.7-5	jsonlite_2.0.0
[49]	hms_1.1.4	listenv_1.0.0	gower_1.0.2
[52]	recipes_1.3.3	rematch_2.0.0	parallelly_1.47.0
[55]	glue_1.8.1	codetools_0.2-20	rsample_1.3.2
[58]	stringi_1.8.7	gtable_0.3.6	furrr_0.4.0
[61]	pillar_1.11.1	htmltools_0.5.9	ipred_0.9-15
[64]	lava_1.9.1	R6_2.6.1	evaluate_1.0.5
[67]	lattice_0.22-9	class_7.3-23	Rcpp_1.1.1-1.1
[70]	prodlim_2026.03.11	xfun_0.59	pkgconfig_2.0.3

Acknowledgments

本研究の実施にあたり、公表データ（SOA 10-Year Level Premium Term Lapse Study）を提供いただいた Society of Actuaries (SOA) に深謝いたします。なお、本論文の作成プロセスにおける英文表現の推敲およびコード構築の補助にあたっては、適切な人間による確認と責任のもとで生成 AI ツール（Gemini / ChatGPT 等）を活用いたしました。

References

Kozuma, R., Asashiba, R., Matsumori, Y., & Iwasawa, H. (2026). *Enhancing GLM-based lapse rate models for EV calculation using interpretable machine learning.*